Metanode – Báo cáo kiến trúc tổng thể

Blockchain tương thích EVM: Đồng thuận Mysticeti DAG-BFT (Rust) · Thực thi song song Block-STM (Go) · Máy ảo MVM (C++) · Trạng thái NOMT

1. Tổng quan kiến trúc

Metanode là hệ thống blockchain hiệu năng cao tương thích Ethereum, được thiết kế theo mô hình hai tầng: một **chuỗi cha (Parent Chain –** `cmd/parent_chain`**)** đóng vai trò lớp thanh toán bù trừ (clearing house), quản lý sổ cái tài khoản đại diện cụm (`NodeFloatAccount`) và sổ đăng ký toàn cục; cùng nhiều **cụm thực thi (Execution Cluster –** `cmd/simple_chain`**)** xử lý giao dịch người dùng và hợp đồng thông minh. Trong mỗi node, **Go** là tiến trình chủ quản lý mạng thực thi, mempool, Block-STM và trạng thái, còn lõi đồng thuận **Rust** được biên dịch thành thư viện (`libmetanode`) liên kết trực tiếp qua CGo FFI trong cùng một tiến trình duy nhất.

- **Đồng thuận BFT (Mysticeti DAG-BFT):** Viết bằng Rust (`consensus/metanode`, `meta-consensus/core`), dung sai lỗi Byzantine $f = \lfloor (n-1)/3 \rfloor$; một cụm BFT cần tối thiểu $n \geq 4$ validator để chịu được 1 node lỗi.
- **Hai chế độ vận hành cụm thực thi:** `consensus_mode = "bft"` (các validator chạy Mysticeti DAG-BFT) hoặc `consensus_mode = "raft"` (tắt Rust consensus, dùng bộ tuần tự hoá `hashicorp/raft` tại `pkg/rollup/raftfeed`).
- **Thực thi song song xác định:** Gom nhóm giao dịch bằng Union-Find (`pkg/grouptxns`), thực thi song song lạc quan bằng Block-STM (`true_block_stm.go` + `mvcc/`) kết hợp thực thi tuần tự rào chắn (Barrier Tx) cho các giao dịch cổng hệ thống.
- **Nguyên tắc an toàn tối thượng (Zero-Fork Invariant):** Thà chờ (PENDING) chứ tuyệt đối không fork; chỉ giao (dispatch) commit sang Go khi có bằng chứng dữ liệu từ 2f+1 peer đồng ý trên cùng digest; nghiêm cấm dùng `timeout/sleep` để thoát deadlock.

1.1. Các lớp kiến trúc trong một Node

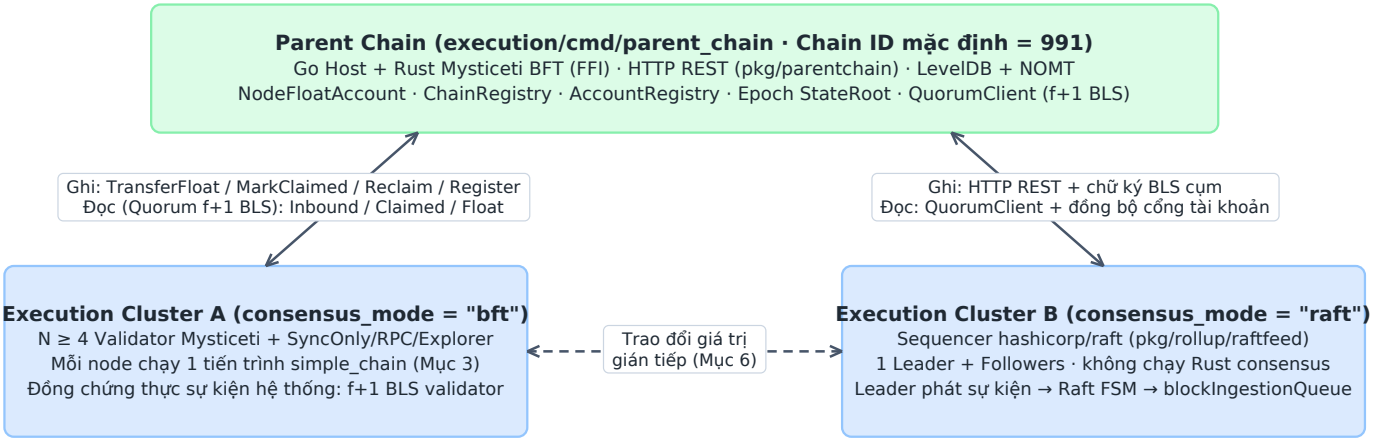
Lớp	Công nghệ & Thư mục nguồn	Vai trò cốt lõi
Đồng thuận	Rust – `consensus/metanode`, `meta-consensus/core`	Đề xuất block DAG, `Linearizer` sắp xếp thứ tự commit tất định, `CommitVoteMonitor` kiểm tra quorum 2f+1 digest, `CommitSyncer` đồng bộ CertifiedCommit, `liveness_watchdog`
Thực thi	Go – `execution/cmd/simple_chain`, `execution/pkg/`	Cổng JSON-RPC/WS (`eth_*`, `mtn_*`) & Protobuf TCP, Mempool, gom nhóm Union-Find, Block-STM/MVCC, Barrier Tx, đồng bộ P2P, snapshot, pruning
Máy ảo	C++ – `execution/pkg/mvm` (CGo)	Máy ảo MVM nhúng tương thích EVM, mô hình gas Ethereum (EIP-150/161/2929/3529…), precompile chuẩn 1–10, BLS12-381 và Xapian
Trạng thái & DB	Rust NOMT (`pkg/nomt_ffi`) + PebbleDB + C++ Xapian + RocksDB	2 cây Merkle nhị phân xác thực NOMT (`account_db_nomt`, `stake_db_nomt`), 7 DB PebbleDB lưu block/tx/receipt/state, Xapian đánh chỉ mục toàn văn, RocksDB lưu DAG theo epoch
Chuỗi cha & Rollup	Go + Rust – `cmd/parent_chain`, `pkg/parentchain`, `pkg/rollup`	Đăng ký cụm (`ChainRegistry`) & tài khoản (`AccountRegistry`), thanh toán bù trừ `NodeFloatAccount`, neo `StateRoot`, đọc xác thực qua `QuorumClient` (f+1 BLS)

1.2. Phân loại vai trò Node (`simple_chain`) và Cấu trúc lưu trữ (`data/`)

Mọi node trong cụm thực thi dùng chung một binary `simple_chain` và phân tách vai trò qua cấu hình: **Validator** (đồng thuận BFT/Raft và thực thi block), **RPC / SyncOnly** (không bỏ phiếu, đồng bộ qua `rust_sync_node` để phục vụ truy vấn), và **Explorer** (bật `EnableExplorerSearch=true`). Kiến trúc lưu trữ mới dùng trực tiếp cây xác thực NOMT và đã loại bỏ hoàn toàn các bảng `changelog_db_*` cũ, giảm tới 50% Disk I/O khi commit block.

Thư mục DB trên đĩa	Động cơ lưu trữ	Validator	SyncOnly	Explorer	Nội dung lưu trữ & Mục đích
`block_db/`, `transaction_db/`, `receipt_db/`	PebbleDB (Go)	Có	Có	Có	Block header/body, chỉ mục chiều cao ↔ hash, giao dịch gốc, biên lai thực thi (`Receipt`) và `EventLogs`
`account_db/` + `account_db_nomt/`	PebbleDB + NOMT	Có	Có	Có	Số dư, nonce, cờ đăng ký chuỗi cha (`ParentRegistered`, `Provisional`, `Locked`) và cây Merkle nhị phân tính `stateRoot` / `eth_getProof`
`stake_db/` + `stake_db_nomt/`	PebbleDB + NOMT	Có	Có	Có	Cổ phần Validator, thông tin uỷ quyền (Delegation), danh sách uỷ ban (Committee) theo epoch và cây xác thực NOMT tương ứng
`code_db/`, `smart_contract_db/`	PebbleDB (Go)	Có	Có	Có	Bytecode hợp đồng thông minh (`codeHash` → bytecode), các ô nhớ EVM (`Storage Slots`) và bảng trạng thái Rollup nội cụm
`xapian_db/`, `snapshot_data/`	C++ Xapian / File	Có	Có	Có	CSDL tìm kiếm toàn văn cho hợp đồng dùng Xapian precompile; các bản chụp trạng thái phục vụ đồng bộ nhanh (State/Commit Sync)
`explorer_search_db/`	C++ Xapian	Tắt	Tắt	Bật	Chỉ mục toàn văn cho Block, Tx, Address, EventLogs (gom mẻ `Commit()` mỗi 64 block qua CGo; tắt trên Validator để tối ưu CPU/I/O)
`storage/node_X/epoch_Y/`	RocksDB (Rust)	Có (BFT)	Có (Sync)	Có (Sync)	Lưu trữ bền vững các block DAG, chứng chỉ đồng thuận và trạng thái vòng (round) theo từng epoch của tầng Rust Mysticeti

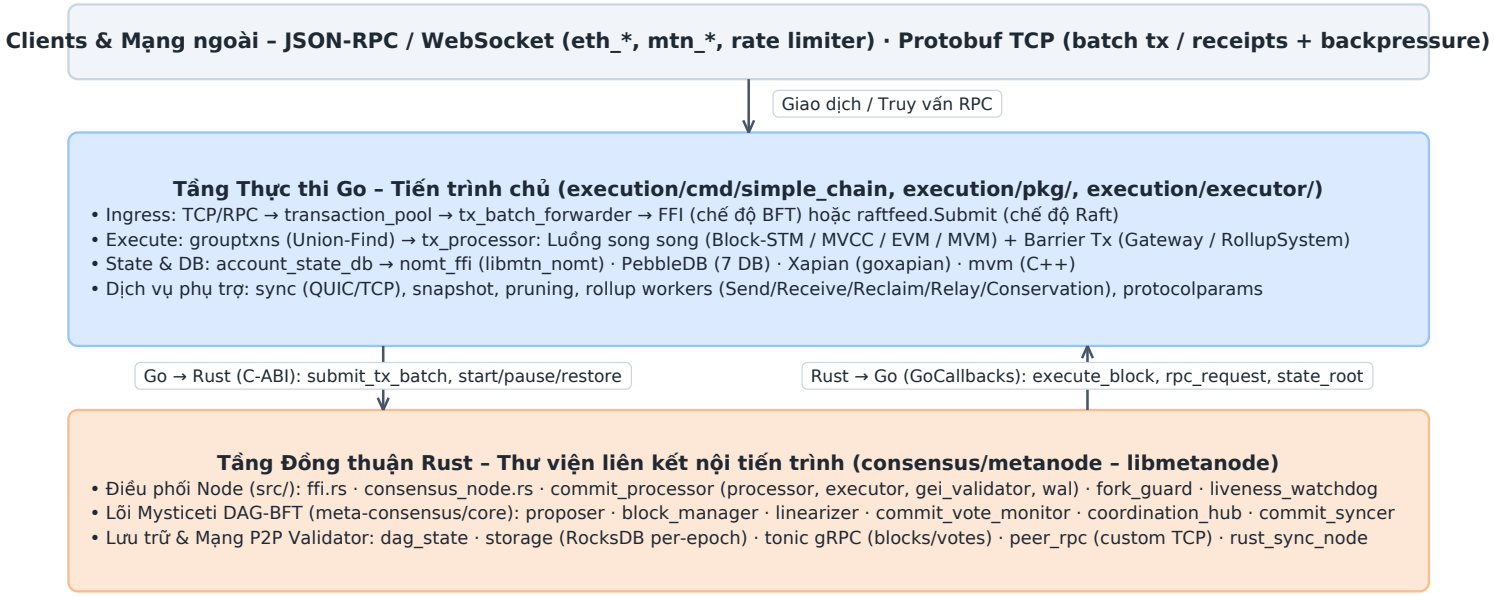
2. Topology hệ thống đa cụm và Chuỗi cha



Hình 1 – Topology tổng thể: Các cụm thực thi không kết nối P2P trực tiếp với nhau mà thanh toán bù trừ và đồng bộ danh tính tài khoản thông qua Chuỗi cha (Chain ID lấy từ genesis, mặc định 991). Mọi thao tác đọc trạng thái từ Chuỗi cha về cụm đều đi qua QuorumClient (đối chiếu f+1 node).

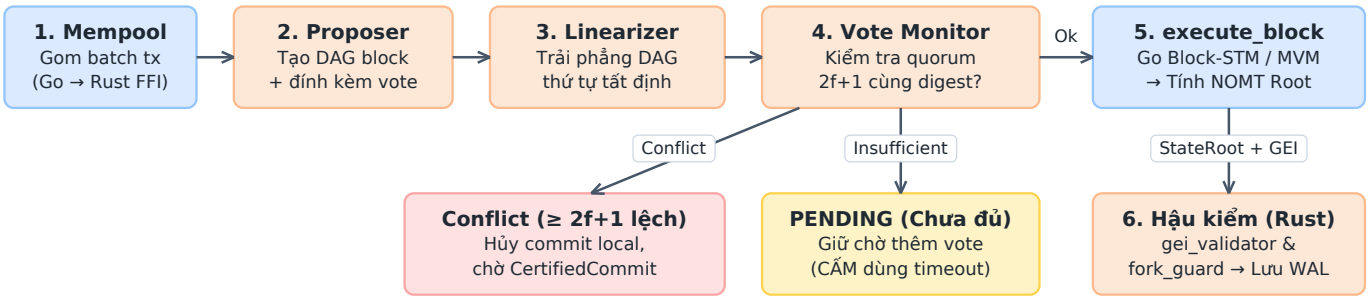
- **Cụm BFT (`consensus_mode = "bft"`):** Các validator chạy đồng thuận Mysticeti DAG-BFT, thực thi cùng chuỗi commit đã xác nhận quorum 2f+1 và sinh ra cùng `StateRoot`. Khi áp dụng sự kiện Rollup/đăng ký tài khoản từ Chuỗi cha vào cụm có nhiều validator, `RollupSystemHandler` bắt buộc gói sự kiện phải có đủ f+1 chữ ký BLS của uỷ ban validator hiện hành (cơ chế fail-closed).
- **Cụm Raft (`consensus_mode = "raft"`):** Tầng Rust consensus không khởi chạy; gói `pkg/rollup/raftfeed` dùng `hashicorp/raft` để nhân bản log và tạo `ExecutableBlock` giống hệt nhau trên mọi replica rồi đẩy thẳng vào `blockIngestionQueue`.
- **Chuỗi cha (`cmd/parent_chain`):** Giữ số dư đại diện (`NodeFloatAccount`) của từng cụm. Tổng số dư mọi tài khoản trên một cụm luôn khớp với số dư Float của cụm đó trên Chuỗi cha (trừ phần đang bay có kiểm soát bởi `ConservationGuard`).

3. Phân lớp kiến trúc một Node (tiến trình `simple_chain`)



Hình 2 – Phân lớp nội bộ một tiến trình simple_chain: Go làm tiến trình chủ, gọi xuống Rust qua C-ABI và nhận lệnh thực thi block / truy vấn dữ liệu ngược lại từ Rust qua bảng hàm con trỏ GoCallbacks (executor/ffi_bridge.go ↔ src/ffi.rs), hoàn toàn không qua socket mạng nội bộ.

4. Luồng Commit và Cơ chế Chống Phân Nhánh (Zero-Fork)



Hình 3 – Luồng kiểm duyệt commit (Zero-Fork Path): CommitVoteMonitor chỉ cho phép gọi execute_block sang Go khi có bằng chứng dữ liệu 2f+1 validator đồng ý trên cùng digest (hoặc trạng thái Cold-start thật). Nếu chưa đủ phiếu, commit giữ nguyên trạng thái PENDING.

4.1. Ma trận quyết định đồng thuận (`PeerAttestResult`)

Điều kiện quan sát tại CommitVoteMonitor	Kết quả	Hành động tại CommitProcessor
Không có dữ liệu digest cục bộ và chưa có peer nào bỏ phiếu cho index này	`Ok`	**Cold-start thật** (mọi node mới khởi tạo epoch cùng DAG giống hệt nhau) → cho phép dispatch sang Go
Không có dữ liệu digest cục bộ nhưng đã ghi nhận phiếu bầu từ peer	`Insufficient`	**Cold-start một phần** (các peer đang bắt đầu bỏ phiếu) → giữ `PENDING`, chờ thu thập đủ phiếu
Đạt tối thiểu `2f+1` peer bỏ phiếu cùng digest với commit cục bộ	`Ok`	**Quorum đã xác nhận** → ghi WAL và gọi FFI `execute_block` sang tầng thực thi Go
Đã có phiếu bầu từ peer nhưng số lượng cùng digest chưa đạt `2f+1`	`Insufficient`	Chưa đủ bằng chứng quorum → giữ `PENDING`, vòng lặp propose vẫn chạy để tích luỹ thêm phiếu
Có `2f+1` peer bỏ phiếu cho một digest khác với digest cục bộ	`Conflict`	Commit cục bộ bị lệch → **huỷ bỏ commit cục bộ**, chờ tải `CertifiedCommit` chuẩn từ `CommitSyncer`

4.2. Bốn nguyên tắc bất biến bảo vệ an toàn trạng thái

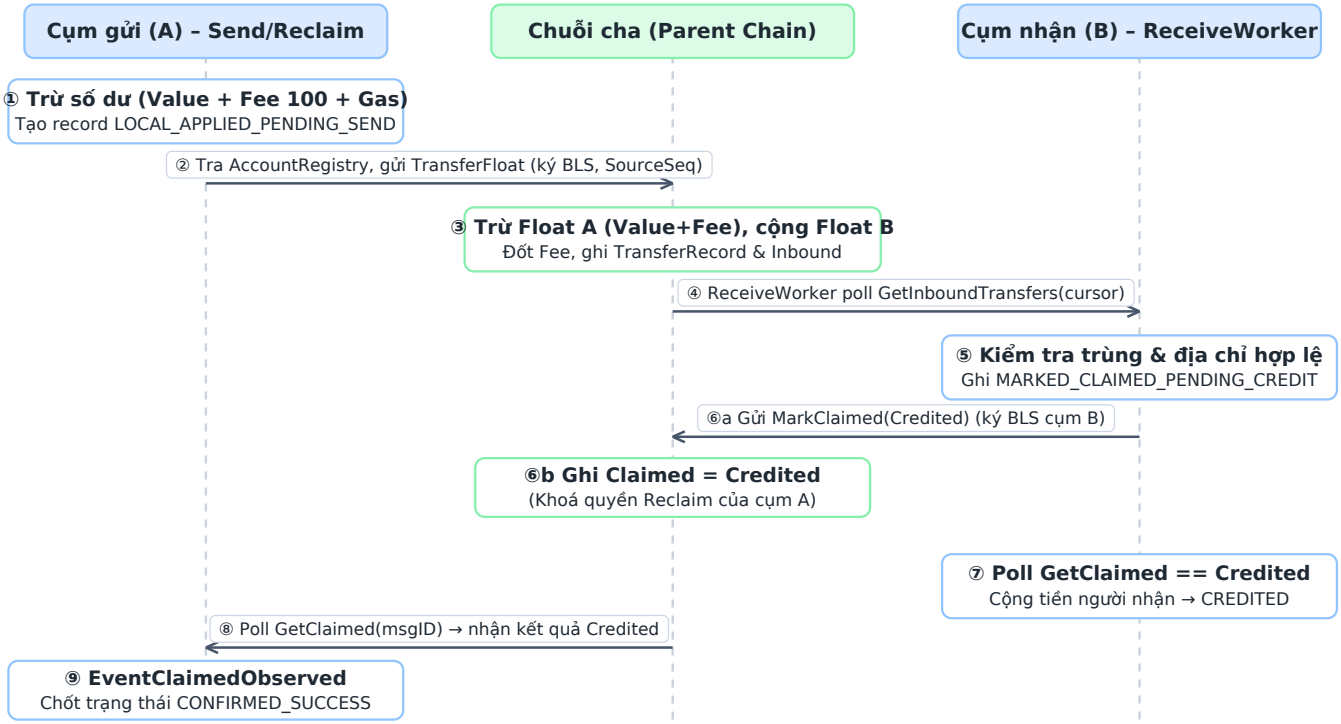
- **Tuyệt đối không dùng Timeout để quyết định Dispatch:** Không dùng `sleep()`, `timeout()` hay `Duration` để tự động đẩy commit đang chờ, vì độ trễ đồng hồ khác nhau giữa các node sẽ làm thứ tự dispatch lệch nhau dẫn tới fork.
- **Giải quyết Deadlock thuần bằng dữ liệu P2P (`CommitVoteMonitor`):** Việc đề xuất DAG block mới không phụ thuộc vào việc commit cũ đã thực thi xong hay chưa; miễn có ≥ `2f+1` node hoạt động, các block mới luôn mang theo `commit_votes` giúp hệ thống tự tiến triển.
- **Dừng an toàn thay vì đoán dữ liệu (Halt-rather-than-guess):** Khi thiếu payload giao dịch mà không peer nào cung cấp được, Rust dừng giao commit (`CONSENSUS-HALT-TX-PAYLOAD-LOST`). Nếu bộ lập lịch Tokio bị treo, `liveness_watchdog.rs` chủ động dừng tiến trình để bảo toàn DB.
- **Đối chiếu hậu thực thi (`gei_validator` & `fork_guard`):** Sau mỗi block thực thi ở Go, chỉ số thực thi toàn cục (`GlobalExecIndex - GEI`) và `StateRoot` NOMT được trả về Rust để kiểm chứng chéo trước khi chuyển sang commit kế tiếp.

5. Động cơ Thực thi, Máy ảo và Quản lý Trạng thái

Thành phần	Mã nguồn chính	Cơ chế hoạt động chi tiết
Gom nhóm giao dịch	`pkg/grouptxns/`, `pkg/transaction_grouper/`	Phân tích tập địa chỉ đọc/ghi của từng giao dịch bằng cấu trúc **Union-Find** để chia thành các nhóm độc lập; thứ tự phẳng hoá sau gom nhóm là tất định trên mọi validator.
Thực thi song song Block-STM	`pkg/blockchain/tx_processor/true_block_stm.go`, `mvcc/`, `native_fast_path.go`	Thực thi lạc quan đa luồng trên bộ nhớ đa phiên bản (MVCC), tự động phát hiện xung đột đọc-ghi giữa các tx trong nhóm để chạy lại (re-execute), kèm đường nhanh cho chuyển khoản gốc.
Giao dịch rào chắn (Barrier Tx)	`parent_chain_gateway_handler.go`, `rollup_system_handler.go`, `gateway_handler.go`	Các giao dịch hệ thống, đăng ký tài khoản và chuyển tiền liên cụm được thực thi tuần tự trên snapshot `chainState` trực tiếp của block đang chạy, đảm bảo tính nguyên tử của nonce và số dư.
Máy ảo MVM (C++)	`pkg/mvm/` (liên kết CGo)	Máy ảo EVM nhúng viết bằng C++: tuân thủ biểu phí gas Ethereum (EIP-150/161/2929/3529…), khởi tạo nonce hợp đồng chuẩn EIP-161, hỗ trợ precompile 1–10, BLS12-381 và Xapian; đã kiểm thử đối chiếu vi sai với `go-ethereum`.
Cây trạng thái NOMT	`pkg/account_state_db/`, `pkg/nomt_ffi/` (`libmtn_nomt`)	Thay thế MPT truyền thống bằng **Nearly Optimal Merkle Trie (NOMT)** viết bằng Rust, tính toán `StateRoot` song song và tạo bằng chứng Merkle (`eth_getProof`) trực tiếp từ trạng thái mới nhất.
Đồng bộ, Snapshot & Fingerprint	`pkg/sync/`, `pkg/snapshot/`, `pkg/pruning/`, `pkg/protocolparams/`	Đồng bộ block qua QUIC + TCP tuỳ biến; khôi phục nhanh qua `snapshot_manager.go`; `protocolparams` kiểm tra vân tay tham số thực thi khớp với genesis ngay khi khởi động (`mtn_protocolFingerprint`).

6. Giao dịch trao đổi giữa 2 cụm thực thi và Cổng tài khoản

Hai cụm thực thi không trao đổi P2P trực tiếp mà thanh toán qua **Chuỗi cha** (`pkg/parentchain/` và `pkg/rollup/`). Trước khi tài khoản được phép gửi giao dịch liên cụm (khi bật **Account Gate** mặc định tại `ParentChainGatewayHandler`), địa chỉ đó phải được đăng ký với Chuỗi cha qua `RegistrationRelay` (`NONE → PENDING → PROVISIONAL → CONFIRMED`; nếu đã đăng ký ở cụm khác trước đó thì bị đánh dấu `REJECTED / Locked`). Tài khoản đang ở trạng thái tạm thời (`PROVISIONAL`) hoặc bị khoá (`Locked`) sẽ bị chặn ngay tại cổng `0x...1003`.



Hình 4 – Luồng chuyển giá trị liên cụm thành công (A → Chuỗi cha → B): Thực hiện bởi ParentChainGatewayHandler + SendWorker ở cụm A và ReceiveWorker + RollupSystemHandler ở cụm B. Mọi bước chuyển trạng thái nội cụm đều đi qua giao dịch hệ thống tất định.

6.1. Các nhánh xử lý ngoại lệ và cơ chế bảo vệ (`pkg/rollup/statemachine.go`)

Tình huống / Cơ chế	Quy trình xử lý tất định trong máy trạng thái & Chuỗi cha
Địa chỉ nhận không hợp lệ (`!IsDestinationValid`)	Cụm B ghi `MARKED_CLAIMED_PENDING_REFUND` → gửi `MarkClaimed(Refund)` lên Chuỗi cha → sau khi xác nhận, cụm B phát `EventClaimedConfirmed(Refund)` để gửi giao dịch `TransferFloat(..., isRefund=true)` hoàn trả `Value` (miễn phí, không tính vào velocity) về cụm A (`REFUND_SENT → REFUNDED`). Cụm A thấy `Outcome=Refund` chuyển sang `REFUND_IN_TRANSIT → CONFIRMED_REFUNDED`.
Trùng MessageID (`IsDuplicate`)	Nếu gặp lần đầu ở `StateNone` → chuyển sang `SKIPPED_DUP`. Nếu `ReceiveWorker` quét lại một bản ghi đã vượt qua `StateNone` (ví dụ sau khi khởi động lại node vì con trỏ `cursor` chỉ giữ trên RAM) → xử lý idempotent no-op, không bao giờ cộng tiền hai lần.
Cụm B không claim sau 60s (`ReclaimWorker`)	Sau khi quá 60s kể từ `ConfirmedAtBlockTime`, `ReclaimWorker` ở A gửi `ReclaimFloat` (ký BLS). Chuỗi cha kiểm tra `blockTime - ConfirmedAtBlockTime ≥ 60s` (dùng thời gian block Chuỗi cha, không dùng đồng hồ node): nếu chưa bị claim, Chuỗi cha trừ `Value` khỏi Float B, hoàn vào Float A và đặt `Claimed=Reclaimed`; cụm A phát `EventReclaimWon` hoàn tiền cho người gửi (`CONFIRMED_REFUNDED`).
Bảo toàn tổng cung cụm (`ConservationGuard`)	Đo đạc bất biến `0 ≤ Float(BLS) − Sum(Accounts) ≤ Pending` khi tập giao dịch đang bay ổn định (`Stable=true`). Nếu phát hiện vi phạm hoặc mất kết nối xác minh quá ngưỡng, cổng `Gate` lập tức khoá `SendWorker` (chặn chuyển tiền mới ra khỏi cụm), nhưng vẫn cho phép nhận và hoàn tất các giao dịch đang bay.
Giới hạn lưu lượng rút 24h (`VelocityLimit`)	Trên Chuỗi cha (`checkAndRecordFloatVelocity`), tổng giá trị chuyển đi (không tính `isRefund`) của mỗi cụm trong cửa sổ trượt 24 giờ bị giới hạn ở mức mặc định **20%** số dư Float đầu cửa sổ, ngăn chặn rủi ro rút cạn thanh khoản đột ngột.
Đồng thuận sự kiện nội cụm (BFT vs Raft)	Ở cụm **Raft**: chỉ Leader phát sự kiện qua `EventProposer → raftfeed → Raft FSM`. Ở cụm **BFT đa validator**: mọi sự kiện hệ thống và đăng ký tài khoản gửi tới `RollupSystemAddress` bắt buộc đính kèm đủ `f+1` chữ ký BLS của uỷ ban validator (`ApplyAttestedSystemEvent`).

Lưu ý kiến trúc: Ngoài luồng thanh toán qua Chuỗi cha ở trên, mã nguồn còn hỗ trợ mô hình cổng Cross-Chain riêng cho các mạng Private Chain (`pkg/cross_chain/`: `GatewayPrecompile`, `Root Anchor / Reserve`, `relayer_daemon` phi tập trung và chứng thực chữ ký BLS tổng hợp của uỷ ban).

7. Kênh giao tiếp giữa các lớp (Cross-Layer Channels)

Kênh giao tiếp	Hướng	Giao thức / Cơ chế	File / Module nguồn tiêu biểu
Giao commit để thực thi	Rust → Go	FFI callback `execute_block` → `cgo_execute_block`	`node/executor_client/block_sending.rs` → `executor/ffi_bridge.go`
RPC nội bộ (epoch, block, GEI, sync)	Rust → Go	FFI callback `process_rpc_request` (Protobuf)	`rpc_queries*.rs` → `executor/unix_socket_handler*.go`
State root / tx trace / log	Rust → Go	FFI callbacks `get_state_root`, `update_tx_trace`, `log_message`	`consensus/metanode/src/ffi.rs` ↔ `executor/ffi_bridge.go`
Chuyển tiếp batch giao dịch	Go → Rust	C-ABI `metanode_submit_transaction_batch`	`tx_batch_forwarder_core.go` → `ffi.rs` → `tx_socket_server.rs`
Vòng đời & điều khiển đồng thuận	Go → Rust	C-ABI `start_consensus`, `register_callbacks`, `pause/resume`, `restore_from_snapshot`, `attest_payload_loss*`	`executor/ffi_bridge.go` → `consensus/metanode/src/ffi.rs`
Truy cập Trie & Merkle Proof	Go → Rust NOMT	C-ABI `libmtn_nomt` (CGo)	`pkg/nomt_ffi/` → `pkg/nomt_ffi/rust_lib`
Nguồn tạo block chế độ Raft	Go nội bộ	`hashicorp/raft` → FSM → `blockIngestionQueue`	`pkg/rollup/raftfeed/` (bỏ qua Rust consensus)
Quảng bá DAG block & vote	Rust ↔ Rust	Mạng `tonic` gRPC bất đồng bộ	`meta-consensus/core/src/network/`
Chứng thực digest commit	Rust ↔ Rust	Phiếu `commit_votes` nhúng trực tiếp trong DAG block	`coordination_hub.rs` ↔ `commit_vote_monitor.rs`
Peer RPC (ranh giới epoch, sync)	Rust ↔ Rust	Giao thức TCP tuỳ biến	`network/peer_rpc/server.rs` ↔ `client.rs`
Đồng bộ dữ liệu block thực thi	Go ↔ Go	QUIC + TCP tuỳ biến (kèm kiểm soát luồng)	`pkg/network/`, `pkg/quic_network/`
Thanh toán & Đăng ký liên cụm	Cụm ↔ Chuỗi cha	HTTP REST + chữ ký BLS / `QuorumClient` (f+1 BLS)	`pkg/parentchain/`, `pkg/rollup/`

8. Vùng thay đổi rủi ro cao (High-Risk Change Zones)

Khi sửa đổi bất kỳ module nào trong bảng dưới đây, bắt buộc phải phân tích phạm vi ảnh hưởng (blast radius) trên toàn bộ luồng gọi Go/Rust, tuân thủ tuyệt đối nguyên tắc **Zero-Fork** và chạy kiểm tra biên dịch sạch (`consensus/metanode/scripts/build_check.sh`).

Vùng nhạy cảm	Vị trí file / package trong mã nguồn	Rủi ro tiềm ẩn nếu thay đổi sai
Commit State Root	`execution/pkg/blockchain/block_state_commit.go`	**Fork** – lệch StateRoot giữa các node sau khi thực thi block
Pipeline trạng thái tài khoản	`execution/pkg/account_state_db/account_state_db_commit.go`	**Fork** – sai lệch thứ tự ghi hoặc băm nút lá trên cây NOMT
Luật thực thi giao dịch	`execution/pkg/blockchain/tx_processor/` (Block-STM, fast path, bộ lọc phí/chữ ký, Barrier Tx)	**Fork** – thay đổi luật đồng thuận; bắt buộc nâng cấp đồng loạt mọi validator
Xử lý đồng bộ Peer (Go)	`execution/cmd/simple_chain/processor/block_processor_sync.go`	**Lệch trạng thái (State divergence)** giữa node đồng bộ và validator
Cầu giao tiếp FFI Go ↔ Rust	`pkg/nomt_ffi/`, `executor/ffi_bridge.go`, `consensus/metanode/src/ffi.rs`, `executor_client/`	**Crash / lỗi bộ nhớ / deadlock** giữa runtime Go và Tokio Rust
Chuyển giao Epoch	`processor/block_processor_epoch.go`, `src/consensus/epoch_transition.rs`	**Mất dữ liệu hoặc kẹt chuyển tiếp** giữa hai uỷ ban validator khi đổi epoch
Bộ xử lý Commit (Rust)	`consensus/metanode/src/consensus/commit_processor/`	**Sai thứ tự commit** hoặc phá vỡ cổng kiểm tra quorum 2f+1 digest
Bộ sắp xếp tuyến tính (Linearizer)	`consensus/metanode/meta-consensus/core/src/linearizer/mod.rs`	**Fork** – thứ tự trải phẳng DAG không tất định giữa các node
Bộ đồng bộ Commit (CommitSyncer)	`consensus/metanode/meta-consensus/core/src/commit_syncer/mod.rs`	**Lỗi đồng bộ / dữ liệu cũ (stale data)** khi node bắt kịp mạng
Khoá đồng bộ trong Consensus Core	`GLOBAL_TX_CACHE`, `dag_state` (`parking_lot`)	**Treo toàn bộ runtime (Runtime freeze)** nếu giữ khoá xuyên điểm chờ
Bộ chuyển tiếp Batch giao dịch	`execution/cmd/simple_chain/processor/tx_batch_forwarder_core.go`	**Mất giao dịch (Tx loss)** hoặc tắc nghẽn đường vào đồng thuận
Lập lịch tạo khối (Mining / PoH)	`execution/pkg/mining/`, `execution/pkg/poh/`	**Suy giảm thời gian tạo khối (Timing regression)** và giảm thông lượng
Quản lý Snapshot	`execution/executor/snapshot_manager.go`	**Hỏng dữ liệu (Data corruption)** khi sao lưu hoặc khôi phục trạng thái node

9. Hiệu năng đo thực tế

Số liệu dưới đây **đo trực tiếp** bằng `secp_tps_blast` (đếm tx thật trong block, không ngoại suy) trên bản 44bedbcb. Dữ liệu thô: `note/evidence/tps_20261024/*.json, *.log` (kèm sha256 của binary đo).

Điều kiện đo: 4 validator + 1 node đồng bộ **cùng một máy** (104 lõi, 188 GB RAM), bộ sinh tải chạy trên cùng máy, gửi qua TCP, giao dịch chuyển tiền EIP-1559. Đây không phải môi trường nhiều máy; số trên máy khác hoặc mạng thật có thể khác.

TPS end-to-end = số tx được đếm trong block / (thời điểm thấy block cuối − thời điểm bắt đầu gửi). **Cửa sổ commit** = cùng số tx / khoảng từ block đầu đến block cuối có tx của tải.

Burst 25.000 tx gửi một lần (3 lần chạy)

Lần	TPS end-to-end	TPS cửa sổ commit	Số block chứa tx	Tx/block lớn nhất	Mất tx	Zero-Fork
1	10.746,8	18.509,9	4	13.086	0	OK
2	9.797,5	17.532,7	4	11.000	0	OK
3	9.520,2	27.067,1	3	13.000	0	OK

Trung bình TPS end-to-end 3 lần: ≈ 10.021,5 tx/s [TÍNH: trung bình cộng 3 số trên]. Mỗi lần tổng thời gian end-to-end chỉ 2,3–2,6 giây, nên đây là đỉnh ngắn hạn, không phải thông lượng duy trì.

Tải duy trì có kiểm soát tốc độ (60 giây mỗi mức)

Mức gửi	Tx gửi / đã vào block	TPS end-to-end	Block/giây (khi có tải)	Tx/block TB (lớn nhất)	Dồn ứ cuối đợt gửi	Zero-Fork
5.000 tx/s	304.000 / 304.000	5.006,0	1,48	3.416 (6.000)	0	OK
8.000 tx/s	480.000 / 480.000	7.897,9	2,55	3.137 (9.261)	8.000	OK

Ở cả hai mức hệ thống theo kịp tốc độ gửi: không mất tx, thời gian xả dồn ứ cuối đợt 0 s (5.000) và 0,74 s (8.000).

Khả năng đóng block

- Khi có tải, các block chứa tx được thấy cách nhau trung vị 350–500 ms ở hai mức tải duy trì (trung bình 393–677 ms); tốc độ block đo được 1,48–2,55 block/giây tuỳ mức tải. Số tx trong một block tăng theo tải; lớn nhất đo được là 24.000 tx/block (đợt quá tải bên dưới).
- Khi không có tải, trong 30 giây chuỗi không tạo thêm block nào (0 block/giây): hệ thống không sinh block rỗng ở trạng thái nghỉ.

Quá tải không giới hạn tốc độ (kiểm tra giới hạn)

Gửi không giới hạn trong 60 giây dài thực tế tới 1.077 giây (cơ chế gửi bị chậm lại khi cụm không theo kịp; nguyên nhân chi tiết chưa phân tích): gửi 4.842.000 tx, 3.813.000 tx vào block trong cửa sổ đo (đạt 3.522,1 tx/s end-to-end), 1.029.000 tx chưa được đếm khi chạm giới hạn chờ 2 phút. Phần chưa đếm là chưa được chứng minh mất, chỉ là chưa vào block trong cửa sổ đo; chế độ strict của tool báo thất bại đúng như thiết kế. Zero-Fork vẫn OK trên cả 4 validator ở block #1804. Đây là đo khả năng chịu quá tải, không phải TPS duy trì.

So sánh BFT (Mysticeti) và Raft trong cùng điều kiện

Đo bằng cùng một tool, cùng một máy, mỗi lần chạy dựng cụm mới từ bản mẫu (`run_tps_counted.py`, `run_raft_counted.py`). BFT: 4 validator. Raft: 3 replica (không có đồng thuận BFT, mô hình chịu lỗi khác). Số trong ngoặc là các lần chạy riêng lẻ.

Chế độ	Khối lượng	BFT (4 validator)	Raft (3 replica)	Raft / BFT [TÍNH]
Burst	25.000 tx, 3 lần	TB 8.776,2 (8.128,5 – 9.431,6)	TB 15.150,2 (14.700,5; 15.375,0; 15.375,2)	≈ 1,73
Duy trì	≈ 847.000 tx; BFT 3 lần, Raft 2 lần	TB 17.594,9 (17.222,2; 17.460,2; 18.102,2)	TB 25.436,9 (26.154,3; 24.719,6)	≈ 1,45

Tất cả lần chạy: không mất tx, Zero-Fork/parity OK. Burst chỉ kéo dài vài giây nên thấp hơn nhiều so với tải duy trì. Số BFT ở bảng này cao hơn số 11,5k của cụm 4+1 chạy lâu dài ở trên vì khác môi trường (cụm dựng mới, dữ liệu trạng thái nhỏ, không có node đồng bộ thứ 5); không thể đem so trực tiếp hai nguồn số liệu.

Về con số cũ (BFT 14 nghìn, Raft hơn 20 nghìn)

- Hôm nay, cùng khối lượng ≈ 847.000 tx: BFT ≈ 17,6 nghìn tx/s và Raft ≈ 25,4 nghìn tx/s, tức đều cao hơn con số cũ; Raft vẫn nhanh hơn BFT khoảng 1,45 lần.
- Theo ghi chép dự án, TPS phụ thuộc khối lượng tx (giảm khi backlog lớn) và có lần số cũ được tạo ra khi node cũ làm chậm client; vì vậy luôn phải nêu kèm khối lượng, chế độ và cách đếm khi trích dẫn.
- Chưa tìm được lý do để kết luận hệ thống 'giảm hiệu năng' so với trước: số hôm nay không thấp hơn số cũ ở cùng cách đo.

Thử cải thiện BFT: nới giới hạn rate-term của backpressure (kết quả: chưa cải thiện)

Quan sát: trong lúc tải, proposer của Rust cộng thêm khoảng 23 ms vào mỗi vòng dù Go chỉ mất vài chục ms cho một block (rust_rate 5,3 so với go_rate 2,8 /s; commit rỗng không tăng GEI nên tỷ lệ luôn lớn hơn 1). Giả thuyết: bỏ phần trễ này khi Go chưa thật sự tụt hậu sẽ tăng TPS. Đã sửa thử `proposer.rs` (chỉ áp rate-term khi go_lag vượt vùng thoải mái), build sạch, rồi chạy A/B xen kẽ 3 cặp (sustained ≈ 847.000 tx, cụm dựng mới mỗi lần).

Phiên bản	TPS end-to-end từng lần	Trung bình
A (không đổi)	17.222,2; 17.460,2; 18.102,2	17.594,9
B (đã sửa)	17.315,9; 17.796,1; 17.257,3	17.456,5

Hiệu B − A theo cặp: +93,8; +335,9; −844,9, trung bình −138,4, độ lệch chuẩn 623,7 [TÍNH], n = 3: không có cải thiện có ý nghĩa. Giả thuyết không được ủng hộ; thay đổi đã được hoàn tác và không commit. Hotspot CPU khác thấy được trong profile (băm keccak/blake2, hex-encode payload tx khi phát tới peer) là ứng viên cho lần thử tiếp theo (chưa thử, và [GIẢ THUYẾT]: một node dùng vài trăm % CPU trên máy 104 lõi nên có thể độ trễ trên đường tới hạn mới là yếu tố quyết định, không phải tổng CPU).

Giới hạn của phép đo

- Chưa xác định mức tải duy trì tối đa: đã chứng minh được 8.000 tx/s duy trì 60 giây; khoảng từ 8.000 tx/s đến mức quá tải chưa được đo.
- Mỗi cấu hình chỉ chạy 1 lần (burst chạy 3 lần), chưa đủ số lần để kết luận thống kê.
- Chưa đo trên nhiều máy, chưa đo thời gian trễ từng giao dịch (latency) trong báo cáo này.
- Chưa so sánh với bản trước các commit gas/nonce của MVM, nên chưa kết luận ảnh hưởng của mô hình gas mới tới TPS.